

THEORY ASSIGNMENT

Course: Full Stack Web Development
Course Code: 23CM4121
Topic: Express.js Middleware
Student Name: P.Raghu Ram
Roll No.: A24126552102
Date: 21-09-2026

1. QUESTION

What is middleware in Express.js? Explain how middleware works in the request-response cycle. Develop an example of middleware used for logging incoming requests.

2. ANSWER

What is Middleware in Express.js?

In Express.js, **middleware** refers to functions that execute during the lifecycle of a request to the Express server. These functions have access to the request object (*req*), the response object (*res*), and the *next* middleware function in the application's request-response cycle.

Middleware functions can perform the following tasks:

- Execute any generic code or logic.
- Make changes to the request and the response objects (like parsing JSON bodies or adding user auth objects).
- End the request-response cycle by sending a response directly.
- Call the next middleware function in the stack using the *next()* callback.

How Middleware Works in the Request-Response Cycle

When an Express application receives an incoming HTTP request, the request enters a pipeline. The pipeline consists of one or more middleware functions arranged in a stack.

1. The first middleware in the stack receives the *req* and *res* objects. It performs its designated task.
2. Once its task is complete, it **must** either send a response back to the client (ending the cycle) OR call *next()* to pass control to the next middleware in the pipeline.
3. If *next()* is called, the next middleware executes. If a middleware function neither ends the cycle nor calls *next()*, the request will hang indefinitely, resulting in a timeout.

Example: Custom Logging Middleware

Below is a practical example of writing and using a custom middleware function to log the timestamp, HTTP method, and URL of every incoming request before routing it to the appropriate endpoint.

```
const express = require('express');
const app = express();

// 1. Define the Custom Logging Middleware Function
const requestLogger = (req, res, next) => {
  // Capture current time and format it
  const currentTime = new Date().toISOString();

  // Log the details to the console
```

```
console.log(`[${currentTime}] ${req.method} request to ${req.url}`);

// Pass control to the next middleware or route handler in the stack
next();
};

// 2. Register the middleware globally for all incoming requests
app.use(requestLogger);

// 3. Define a basic route handler (which acts as the final middleware)
app.get('/', (req, res) => {
  res.send('Hello World! Your request has been logged.');
```

```
});

// 4. Start the server
app.listen(3000, () => {
  console.log('Server is running on port 3000');
```

```
});
```